

Geometry $\cap$ Rendering: The Point of Parallel Lines

Peter

UNSW Competitive Programming and Mathematics Society

October 6, 2026



Attendance

Please scan the QR code; please let us know about any dietary requirements (for the pizza!).



Introduction

Computer Graphics

Computer graphics (more specifically **good** computer graphics) is pretty much taken for granted nowadays. Ranging from video games, film, animation & art software etc...

Computer Graphics

Computer graphics (more specifically **good** computer graphics) is pretty much taken for granted nowadays. Ranging from video games, film, animation & art software etc...

Majority of work on graphics today is a matter of engineering and optimisation, but curiously, the foundational mathematics behind almost everything range from decades to centuries old.

Computer Graphics

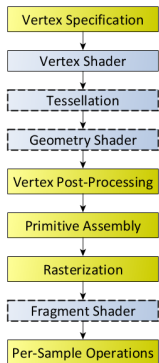
Computer graphics (more specifically **good** computer graphics) is pretty much taken for granted nowadays. Ranging from video games, film, animation & art software etc...

Majority of work on graphics today is a matter of engineering and optimisation, but curiously, the foundational mathematics behind almost everything range from decades to centuries old.

Let's look at various interesting elements of mathematics that build up a good, computer graphics render.

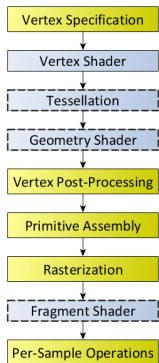
Rendering Pipeline

Modern renderers like OpenGL and Vulkan follow a sequence of steps called a pipeline. We'll not cover everything but hopefully the ones we do cover are interesting.



Rendering Pipeline

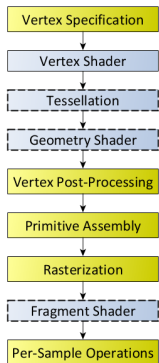
Modern renderers like OpenGL and Vulkan follow a sequence of steps called a pipeline. We'll not cover everything but hopefully the ones we do cover are interesting.



Vertex specification. Sets up an ordered list of vertices to send to the pipeline, which contain useful attributes that'll be used later on.

Rendering Pipeline

Modern renderers like OpenGL and Vulkan follow a sequence of steps called a pipeline. We'll not cover everything but hopefully the ones we do cover are interesting.

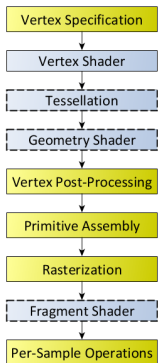


Vertex specification. Sets up an ordered list of vertices to send to the pipeline, which contain useful attributes that'll be used later on.

Vertex shader. Apply basic processing of these vertices, which include transformations, rotations, and most importantly **projection**, allowing 3D objects to be displayed on a 2D screen.

Rendering Pipeline

Modern renderers like OpenGL and Vulkan follow a sequence of steps called a pipeline. We'll not cover everything but hopefully the ones we do cover are interesting.



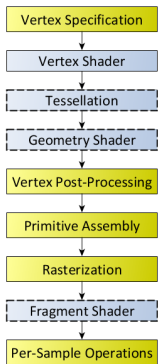
Vertex specification. Sets up an ordered list of vertices to send to the pipeline, which contain useful attributes that'll be used later on.

Vertex shader. Apply basic processing of these vertices, which include transformations, rotations, and most importantly **projection**, allowing 3D objects to be displayed on a 2D screen.

Rasterization. Converts these vertex data to pixels to **fragments**, which contain information like where on the **screen** a pixel is and which pixels need to be shaded.

Rendering Pipeline

Modern renderers like OpenGL and Vulkan follow a sequence of steps called a pipeline. We'll not cover everything but hopefully the ones we do cover are interesting.



OpenGL's Pipeline

Vertex specification. Sets up an ordered list of vertices to send to the pipeline, which contain useful attributes that'll be used later on.

Vertex shader. Apply basic processing of these vertices, which include transformations, rotations, and most importantly **projection**, allowing 3D objects to be displayed on a 2D screen.

Rasterization. Converts these vertex data to pixels to **fragments**, which contain information like where on the **screen** a pixel is and which pixels need to be shaded.

Fragment shader. Determines what colour to shade those fragments, (lighting, textures, etc).

Vertex Specification

Building a World out of Triangles

Before we can draw anything, we need something to draw. Almost every 3D model is a **triangle mesh**: a list of vertices, and a list of triangles joining them.

Building a World out of Triangles

Before we can draw anything, we need something to draw. Almost every 3D model is a **triangle mesh**: a list of vertices, and a list of triangles joining them.

So let's try to build the most fundamental shape of all, a **sphere**. How well can we actually approximate one with triangles?

Building a World out of Triangles

Before we can draw anything, we need something to draw. Almost every 3D model is a **triangle mesh**: a list of vertices, and a list of triangles joining them.

So let's try to build the most fundamental shape of all, a **sphere**. How well can we actually approximate one with triangles?

Surely this will **not** come back to haunt us later.

Graph Theory

Suppose every face of our mesh has p sides and every vertex has degree q .

Graph Theory

Suppose every face of our mesh has p sides and every vertex has degree q .

By counting the same incidences in two ways, we get

$$pF = 2E = qV.$$

Graph Theory

Suppose every face of our mesh has p sides and every vertex has degree q .

By counting the same incidences in two ways, we get

$$pF = 2E = qV.$$

This, combined with Euler's formula, it puts a surprisingly strong restriction on what a perfectly regular mesh can look like.

Only Five?

Since our mesh aims to be some sort of “net” for the sphere, we can represent it as a planar graph and use Euler’s Formula.

Euler’s formula

For any planar graph,

$$V - E + F = 2.$$

Only Five?

Since our mesh aims to be some sort of “net” for the sphere, we can represent it as a planar graph and use Euler’s Formula.

Euler’s formula

For any planar graph,

$$V - E + F = 2.$$

Using $pF = 2E = qV$ and rearranging gives

$$\frac{1}{p} + \frac{1}{q} = \frac{1}{2} + \frac{1}{E} > \frac{1}{2}.$$

Only Five?

Since our mesh aims to be some sort of “net” for the sphere, we can represent it as a planar graph and use Euler’s Formula.

Euler’s formula

For any planar graph,

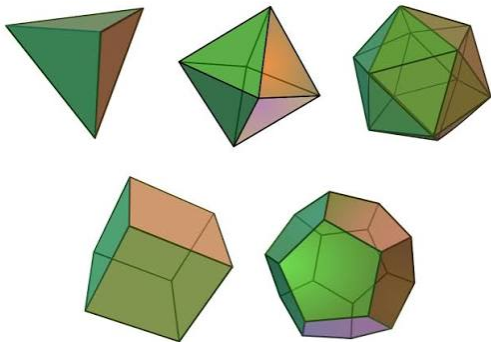
$$V - E + F = 2.$$

Using $pF = 2E = qV$ and rearranging gives

$$\frac{1}{p} + \frac{1}{q} = \frac{1}{2} + \frac{1}{E} > \frac{1}{2}.$$

With $p, q \geq 3$, we are left with only five possibilities.

The 5 Platonic Solids



For our sphere

The icosahedron already gives us **20 triangular faces**, so it is the natural place to start.

Figure: The 5 regular Platonic solids

Icosphere

Remember we're trying to replicate a sphere with **triangles**, so let's use an icosahedron since it has the most triangles.

Icosphere

Remember we're trying to replicate a sphere with **triangles**, so let's use an icosahedron since it has the most triangles.

Start from an icosahedron, we subdivide each triangle into four smaller triangles and “push” each vertex further from the center as to better approximate the shape of a sphere.

Icosphere

Remember we're trying to replicate a sphere with **triangles**, so let's use an icosahedron since it has the most triangles.

Start from an icosahedron, we subdivide each triangle into four smaller triangles and “push” each vertex further from the center as to better approximate the shape of a sphere.

This is known as an **icosphere**, and in fact is a commonly used **mesh approximation** for spheres, as each triangle is roughly the same area, and behaves well geometrically. However, it still isn't perfect.

Twelve Stubborn Vertices

An icosahedron starts with 12 vertices, each with degree 5. Every new vertex we introduce by subdividing has degree 6, and subdividing never changes the degree of an old vertex. So those 12 stay at degree 5 forever.

Twelve Stubborn Vertices

An icosahedron starts with 12 vertices, each with degree 5. Every new vertex we introduce by subdividing has degree 6, and subdividing never changes the degree of an old vertex. So those 12 stay at degree 5 forever.

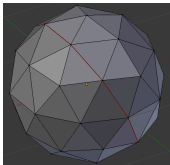
Why does that matter? Since the faces are equilateral triangles, a degree 6 vertex has angles summing to $6 \times 60^\circ = 360^\circ$, which is exactly flat. But a degree 5 vertex only gives $5 \times 60^\circ = 300^\circ$, so it is 60° short.

Twelve Stubborn Vertices

An icosahedron starts with 12 vertices, each with degree 5. Every new vertex we introduce by subdividing has degree 6, and subdividing never changes the degree of an old vertex. So those 12 stay at degree 5 forever.

Why does that matter? Since the faces are equilateral triangles, a degree 6 vertex has angles summing to $6 \times 60^\circ = 360^\circ$, which is exactly flat. But a degree 5 vertex only gives $5 \times 60^\circ = 300^\circ$, so it is 60° short.

This gives us twelve permanent "defects", which is a compromise that **definitely has no consequence whatsoever**.



Icosphere in blender, notice that some vertices have 5 incident edges while others have 6.



Vertex Shader



Adding Another Dimension

Our mesh lives in $\mathbb{R}^3$, but a screen is only $\mathbb{R}^2$. So we'll need mapping from $\mathbb{R}^3$ to $\mathbb{R}^2$ in order to process another dimension.

Note

We'll call the 3D 'world' coordinates P , and the projected 'camera' coordinates P' .



Adding Another Dimension

Our mesh lives in $\mathbb{R}^3$, but a screen is only $\mathbb{R}^2$. So we'll need mapping from $\mathbb{R}^3$ to $\mathbb{R}^2$ in order to process another dimension.

Note

We'll call the 3D 'world' coordinates P , and the projected 'camera' coordinates P' .

Why not just drop depth coordinate?

$$(x, y, z) \mapsto (x, y)$$



Adding Another Dimension

Our mesh lives in $\mathbb{R}^3$, but a screen is only $\mathbb{R}^2$. So we'll need mapping from $\mathbb{R}^3$ to $\mathbb{R}^2$ in order to process another dimension.

Note

We'll call the 3D 'world' coordinates P , and the projected 'camera' coordinates P' .

Why not just drop depth coordinate?

$$(x, y, z) \mapsto (x, y)$$

Because this map is **linear**. Which we **don't** want since a linear map takes parallel lines to parallel lines and preserves ratios along a line. The z coordinate never influences the result, so objects far away are the same size as objects closer to the camera.



Euclid No More

The simple idea that "objects further away are smaller" in fact produces quite a lot of issues as it breaks essentially everything we know about Euclidean geometry:

- Parallel lines are no longer preserved since they now meet



Euclid No More

The simple idea that "objects further away are smaller" in fact produces quite a lot of issues as it breaks essentially everything we know about Euclidean geometry:

- Parallel lines are no longer preserved since they now meet
- Lengths are no longer preserved



Euclid No More

The simple idea that "objects further away are smaller" in fact produces quite a lot of issues as it breaks essentially everything we know about Euclidean geometry:

- Parallel lines are no longer preserved since they now meet
- Lengths are no longer preserved
- Ratios along a line are no longer preserved



Euclid No More

The simple idea that "objects further away are smaller" in fact produces quite a lot of issues as it breaks essentially everything we know about Euclidean geometry:

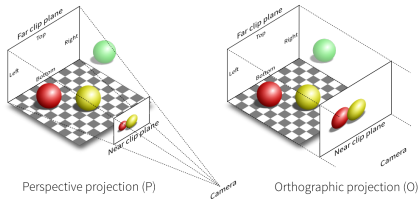
- Parallel lines are no longer preserved since they now meet
- Lengths are no longer preserved
- Ratios along a line are no longer preserved
- Areas, angles, and circles are no longer preserved.



Euclid No More

The simple idea that "objects further away are smaller" in fact produces quite a lot of issues as it breaks essentially everything we know about Euclidean geometry:

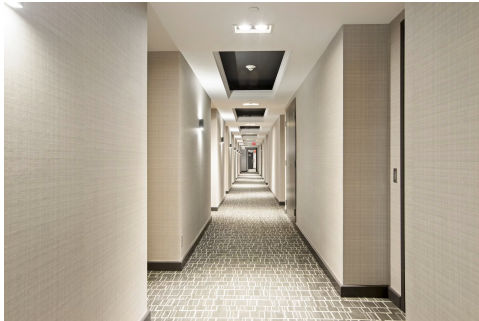
- Parallel lines are no longer preserved since they now meet
- Lengths are no longer preserved
- Ratios along a line are no longer preserved
- Areas, angles, and circles are no longer preserved.



In order for this to work, we need to introduce a new system of geometry.



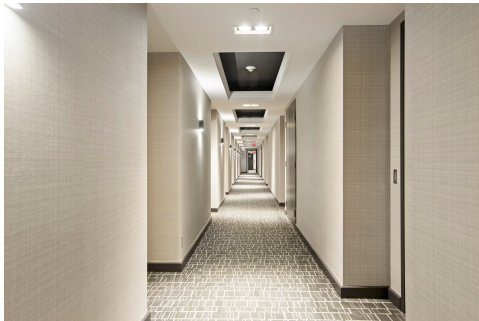
Point At Infinity



Imagine walking down some infinitely long corridor.



Point At Infinity

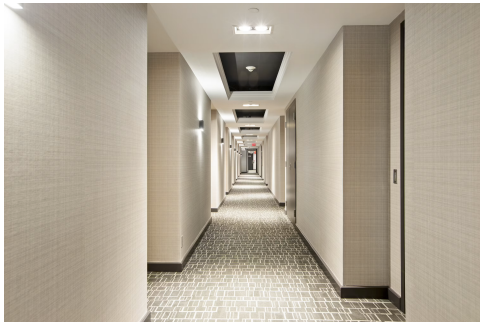


Imagine walking down some infinitely long corridor.

There are parallel lines in the figure, say the two marking the floor. But they are not parallel *in the picture*: they converge towards a point.



Point At Infinity



Imagine walking down some infinitely long corridor.

There are parallel lines in the figure, say the two marking the floor. But they are not parallel *in the picture*: they converge towards a point.

In fact all the parallel lines seem to converge towards **the same point** on the horizon.

This point of convergence called the point at infinity / vanishing point.



The Projective Plane

We're able to formalise this notion of a vanishing point in **real projective plane** ... by introducing a vanishing point to our plane.



The Projective Plane

We're able to formalise this notion of a vanishing point in **real projective plane** ... by introducing a vanishing point to our plane.

- Every direction gets a **point at infinity**, shared by all lines parallel to that direction.



The Projective Plane

We're able to formalise this notion of a vanishing point in **real projective plane** ... by introducing a vanishing point to our plane.

- Every direction gets a **point at infinity**, shared by all lines parallel to that direction.
- All of those points together form a **line at infinity**.



The Projective Plane

We're able to formalise this notion of a vanishing point in **real projective plane** ... by introducing a vanishing point to our plane.

- Every direction gets a **point at infinity**, shared by all lines parallel to that direction.
- All of those points together form a **line at infinity**.

Intersection of Lines

Any two distinct lines now meet at exactly one point. Ordinary lines meet normally, while parallel lines meet at their common point at infinity.

There is a lot more nice geometry in projective planes (especially when dealing with lots of lines) but this is about as much as we need to make perspective useful computationally.



Points as Lines as Points

Remember that visually, objects further away should be smaller, and vice versa, so everything we see lies on an imaginary “line of perspective”.



Points as Lines as Points

Remember that visually, objects further away should be smaller, and vice versa, so everything we see lies on an imaginary “line of perspective”.

So rather than thinking of a projective point as one exact position, we can think of it as an entire **ray through the origin**, and points along that ray are visually equivalent from the camera's point of view.

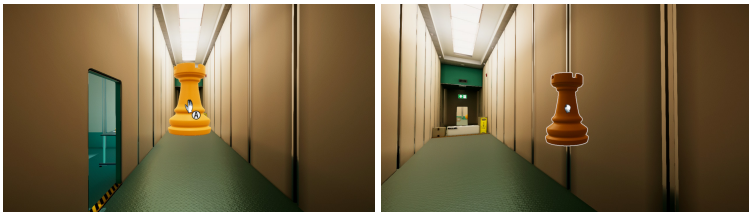


Figure: The game Superliminal is built around this gimmick of perspective.



Fourth Coordinate

To actually enable computation with this “line of perspective”, we actually need to add **another dimension**, you can think of this as the “depth” dimension.



Fourth Coordinate

To actually enable computation with this “line of perspective”, we actually need to add **another dimension**, you can think of this as the “depth” dimension.

Homogeneous Coordinates

We'll begin by writing each point in our 3D space as a 4-tuple (x, y, z, w) , where

$$(x, y, z, w) \sim (\lambda x, \lambda y, \lambda z, \lambda w), \quad \lambda \neq 0.$$

So two points are considered “the same” if they are scalar multiples of each other. These are known as **homogeneous coordinates**.



Fourth Coordinate

To actually enable computation with this “line of perspective”, we actually need to add **another dimension**, you can think of this as the “depth” dimension.

Homogeneous Coordinates

We'll begin by writing each point in our 3D space as a 4-tuple (x, y, z, w) , where

$$(x, y, z, w) \sim (\lambda x, \lambda y, \lambda z, \lambda w), \quad \lambda \neq 0.$$

So two points are considered “the same” if they are scalar multiples of each other. These are known as **homogeneous coordinates**.

Dividing by w picks the representative on the slice $w = 1$:

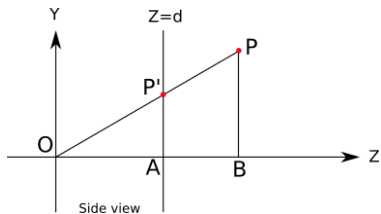
$$(x, y, z, w) \mapsto \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w} \right).$$

Rays with $w = 0$ never meet this slice, and correspond to the **points at infinity**.



Projecting Onto the Screen

Back to the camera. Let's say we put the image plane at $z = d$, with the eye at the origin.



By similar triangles, $P = (x, y, z)$ is drawn at

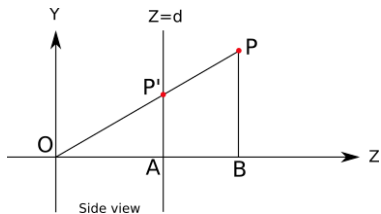
$$P' = \left(\frac{d x}{z}, \frac{d y}{z} \right),$$

so doubling the depth halves the distance from the centre.



Projecting Onto the Screen

Back to the camera. Let's say we put the image plane at $z = d$, with the eye at the origin.



By similar triangles, $P = (x, y, z)$ is drawn at

$$P' = \left(\frac{dx}{z}, \frac{dy}{z} \right),$$

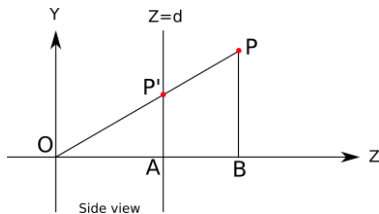
so doubling the depth halves the distance from the centre.

However this requires a **division by a coordinate**, which we can't naively do with matrix multiplication.



Projecting Onto the Screen

Back to the camera. Let's say we put the image plane at $z = d$, with the eye at the origin.



By similar triangles, $P = (x, y, z)$ is drawn at

$$P' = \left(\frac{dx}{z}, \frac{dy}{z} \right),$$

so doubling the depth halves the distance from the centre.

However this requires a **division by a coordinate**, which we can't naively do with matrix multiplication.

Homogeneous coordinates to the rescue

Scaling by $\lambda = z$ doesn't change the point, but it removes the division:

$$\left(\frac{dx}{z}, \frac{dy}{z}, *, 1 \right) \sim (dx, dy, *, z).$$



The Perspective Matrix

Every entry of $(d\ x, d\ y, *, z)$ is linear, so a matrix *can* produce it. We simply have to play z into coordinate w :

$$\begin{pmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & * & * \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} d\ x \\ d\ y \\ * \\ z \end{pmatrix} \sim \begin{pmatrix} d\ x/z \\ d\ y/z \\ * \\ 1 \end{pmatrix}.$$

Converting back out of homogeneous coordinates (dividing by w), then is equivalent to the perspective divide last slide.



The Perspective Matrix

Every entry of $(d\ x, d\ y, *, z)$ is linear, so a matrix *can* produce it. We simply have to play z into coordinate w :

$$\begin{pmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & * & * \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} d\ x \\ d\ y \\ * \\ z \end{pmatrix} \sim \begin{pmatrix} d\ x/z \\ d\ y/z \\ * \\ 1 \end{pmatrix}.$$

Converting back out of homogeneous coordinates (dividing by w), then is equivalent to the perspective divide last slide.

Note on the 3rd coordinate

For us, the important trick is just $w = z$. The GPU then performs the divide by w , which produces perspective. The remaining entries handle things like depth/clipping, which we'll not go through.



Rasterization



Drawing Triangles Well and Good

To have a good triangle drawing algorithm. It should meet the following criteria:

- It should be **simple and fast**.
- It should be **symmetrical**.
- It should be easy for the computer to do ¹

¹Easy to parallelise



Drawing Triangles Well and Good

To have a good triangle drawing algorithm. It should meet the following criteria:

- It should be **simple and fast**.
- It should be **symmetrical**.
- It should be easy for the computer to do ¹

Idea One

We Draw Line by Line

¹Easy to parallelise



Scanline Algorithm

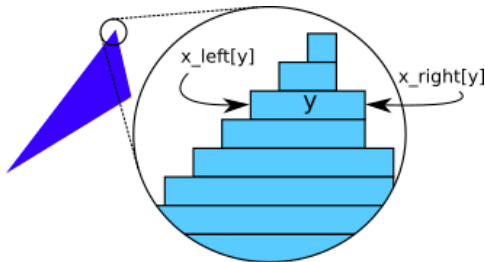


Figure: Decomposing a triangle into horizontal spans

- We'll draw filled triangles by thinking of them as a collection of horizontal line segments that look like a triangle when drawn together.
- Shapes are drawn by sweeping a horizontal line from top (y_{max}) to bottom (y_{min}).



Implementation

1. **Sort Vertices:** Order triangle vertices vertically such that $y_0 \leq y_1 \leq y_2$.



Implementation

1. **Sort Vertices:** Order triangle vertices vertically such that $y_0 \leq y_1 \leq y_2$.
2. **Split Triangle:** Divide the triangle at y_1 into a flat-bottom and flat-top triangle to simplify interpolation.



Implementation

1. **Sort Vertices:** Order triangle vertices vertically such that $y_0 \leq y_1 \leq y_2$.
2. **Split Triangle:** Divide the triangle at y_1 into a flat-bottom and flat-top triangle to simplify interpolation.
3. **Interpolate Edges:** For each scanline y , compute left (x_L) and right (x_R) bounds using line equations:

$$x = x_0 + (y - y_0) \cdot \frac{x_1 - x_0}{y_1 - y_0}.$$



Implementation

1. **Sort Vertices:** Order triangle vertices vertically such that $y_0 \leq y_1 \leq y_2$.
2. **Split Triangle:** Divide the triangle at y_1 into a flat-bottom and flat-top triangle to simplify interpolation.
3. **Interpolate Edges:** For each scanline y , compute left (x_L) and right (x_R) bounds using line equations:

$$x = x_0 + (y - y_0) \cdot \frac{x_1 - x_0}{y_1 - y_0}.$$

4. **Draw Spans:** Fill pixels horizontally from x_L to x_R for the current scanline.



Implementation

1. **Sort Vertices:** Order triangle vertices vertically such that $y_0 \leq y_1 \leq y_2$.
2. **Split Triangle:** Divide the triangle at y_1 into a flat-bottom and flat-top triangle to simplify interpolation.
3. **Interpolate Edges:** For each scanline y , compute left (x_L) and right (x_R) bounds using line equations:

$$x = x_0 + (y - y_0) \cdot \frac{x_1 - x_0}{y_1 - y_0}.$$

4. **Draw Spans:** Fill pixels horizontally from x_L to x_R for the current scanline.

This is messy, creates rounding errors due to interpolation, and is by definition sequential (so no obvious parallelisation). **Can we do better?**



Doing Better with Bounding Boxes

Say if we had some function that allows us to determine whether a point is inside some triangle. Can we design a very straightforward algorithm to draw our triangles? (Hint: read the title)



Doing Better with Bounding Boxes

Say if we had some function that allows us to determine whether a point is inside some triangle. Can we design a very straightforward algorithm to draw our triangles? (Hint: read the title)

Magic

```
1: procedure DRAWTRIANGLE( $P$ )                                ▶  $P$  is 3 points defining the vertices
   FINDBOUNDINGBOX( $P$ )
2:   for each pixel  $p$  in BoundingBox( $P$ ) do
3:     if ISINSIDE( $P, p$ ) then
4:       DRAWPIXEL( $p$ )
5:     end if
6:   end for
7: end procedure
```



Work Dumber Not Smarter

- Not only is the pseudocode much easier on the eyes, it suffers from none of the issues of our **scanline approach**, and is an **embarrassingly parallel** algorithm, meaning it can very easily be divided into subtasks for more processors.



Work Dumber Not Smarter

- Not only is the pseudocode much easier on the eyes, it suffers from none of the issues of our **scanline approach**, and is an **embarrassingly parallel** algorithm, meaning it can very easily be divided into subtasks for more processors.
- **Problem:** How do we actually check if an arbitrary point is within some triangle?



Weighing a Point

Forget triangles for a second. Suppose we put masses $\alpha, \beta, \gamma \geq 0$ at three points A, B, C . Where is the centre of mass?

$$P = \frac{\alpha A + \beta B + \gamma C}{\alpha + \beta + \gamma}.$$



Weighing a Point

Forget triangles for a second. Suppose we put masses $\alpha, \beta, \gamma \geq 0$ at three points A, B, C . Where is the centre of mass?

$$P = \frac{\alpha A + \beta B + \gamma C}{\alpha + \beta + \gamma}.$$

- If we normalise so that $\alpha + \beta + \gamma = 1$, then $P = \alpha A + \beta B + \gamma C$.
- The triple (α, β, γ) is called the **barycentric coordinates** of P with respect to $\triangle ABC$.



Areal Coordinates

Since the coordinates weight the three vertices, it should be unsurprising that they also measure how much of the triangle "belongs" to each vertex, that is, the signed fractions of area it takes up:

$$\alpha = \frac{[PBC]}{[ABC]}, \quad \beta = \frac{[PCA]}{[ABC]}, \quad \gamma = \frac{[PAB]}{[ABC]}.$$

where $[ABC]$ denotes the area of $\triangle ABC$

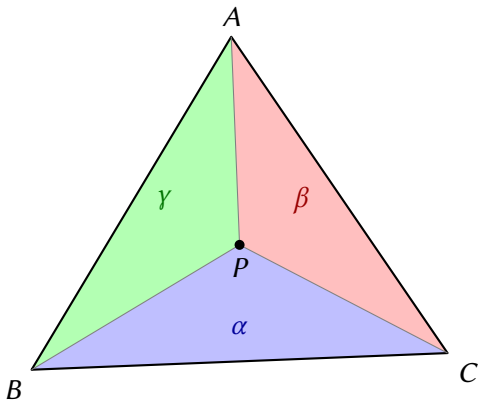


Figure: $P = (\alpha, \beta, \gamma)$ cuts $\triangle ABC$ into three pieces of relative area α, β, γ



Areal Coordinates

Since the coordinates weight the three vertices, it should be unsurprising that they also measure how much of the triangle "belongs" to each vertex, that is, the signed fractions of area it takes up:

$$\alpha = \frac{[PBC]}{[ABC]}, \quad \beta = \frac{[PCA]}{[ABC]}, \quad \gamma = \frac{[PAB]}{[ABC]}.$$

where $[ABC]$ denotes the area of $\triangle ABC$

- α is the "weight" of A : it is 1 at A and 0 along the opposite edge BC .
- What happens to any point P outside the triangle?

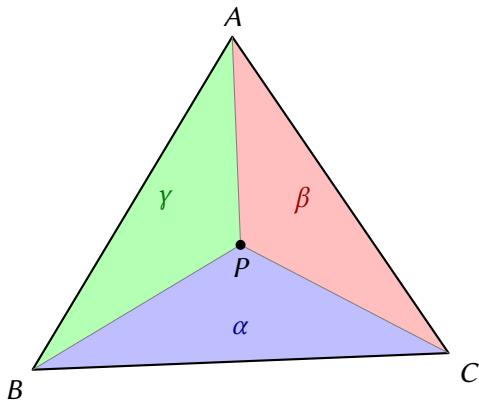


Figure: $P = (\alpha, \beta, \gamma)$ cuts $\triangle ABC$ into three pieces of relative area α, β, γ



Back To Business

Signed Area

A triangle's signed area is positive in one orientation and negative in the other, meaning it can conveniently be calculated using the cross-product:

$$[ABC] = \frac{1}{2} \left(\vec{AB} \times \vec{AC} \right).$$



Back To Business

Signed Area

A triangle's signed area is positive in one orientation and negative in the other, meaning it can conveniently be calculated using the cross-product:

$$[ABC] = \frac{1}{2} \left(\vec{AB} \times \vec{AC} \right).$$

So rather than keeping track of three area fractions, we use the much simpler fact:

Point-in-triangle test

For a point inside the triangle, the three edge tests all have the **same sign**.



Final Algorithm

IsINSIDE

```
1: function IsINSIDE( $(A, B, C), P$ )  
2:    $\alpha \leftarrow \overrightarrow{PB} \times \overrightarrow{PC}, \quad \beta \leftarrow \overrightarrow{PC} \times \overrightarrow{PA}, \quad \gamma \leftarrow \overrightarrow{PA} \times \overrightarrow{PB}$   
3:   return  $(\alpha \geq 0 \wedge \beta \geq 0 \wedge \gamma \geq 0) \vee (\alpha \leq 0 \wedge \beta \leq 0 \wedge \gamma \leq 0)$   
4: end function
```



Final Algorithm

IsINSIDE

```
1: function IsINSIDE((A, B, C), P)
2:    $\alpha \leftarrow \overrightarrow{PB} \times \overrightarrow{PC}, \quad \beta \leftarrow \overrightarrow{PC} \times \overrightarrow{PA}, \quad \gamma \leftarrow \overrightarrow{PA} \times \overrightarrow{PB}$ 
3:   return  $(\alpha \geq 0 \wedge \beta \geq 0 \wedge \gamma \geq 0) \vee (\alpha \leq 0 \wedge \beta \leq 0 \wedge \gamma \leq 0)$ 
4: end function
```

- We can ignore the common denominator since we only care about the sign of the coordinates.
- There are two cases because the vertices may be given clockwise or anticlockwise; all we ask is that the three signs **agree**.
- We no longer need to sort the coordinates and every pixel is decided independently of another.



Final Algorithm

IsINSIDE

```
1: function IsINSIDE( $(A, B, C), P$ )  
2:    $\alpha \leftarrow \overrightarrow{PB} \times \overrightarrow{PC}, \quad \beta \leftarrow \overrightarrow{PC} \times \overrightarrow{PA}, \quad \gamma \leftarrow \overrightarrow{PA} \times \overrightarrow{PB}$   
3:   return  $(\alpha \geq 0 \wedge \beta \geq 0 \wedge \gamma \geq 0) \vee (\alpha \leq 0 \wedge \beta \leq 0 \wedge \gamma \leq 0)$   
4: end function
```

- We can ignore the common denominator since we only care about the sign of the coordinates.
- There are two cases because the vertices may be given clockwise or anticlockwise; all we ask is that the three signs **agree**.
- We no longer need to sort the coordinates and every pixel is decided independently of another.

We're now able to draw triangles. Yippie.



Interpolating Points

Our triangles are now in the right place, the right size, foreshortened correctly, and we know exactly which pixels they cover.



Interpolating Points

Our triangles are now in the right place, the right size, foreshortened correctly, and we know exactly which pixels they cover.

But a triangle is not just three corners. It has to be *filled*, and that means deciding, for every pixel inside it, which part of the texture to paint.



Interpolating Points

Our triangles are now in the right place, the right size, foreshortened correctly, and we know exactly which pixels they cover.

But a triangle is not just three corners. It has to be *filled*, and that means deciding, for every pixel inside it, which part of the texture to paint.

Since we only have data about the vertices of our 3D object, the natural (and really only) thing to do is to interpolate its coordinates.



Interpolating Points

Our triangles are now in the right place, the right size, foreshortened correctly, and we know exactly which pixels they cover.

But a triangle is not just three corners. It has to be *filled*, and that means deciding, for every pixel inside it, which part of the texture to paint.

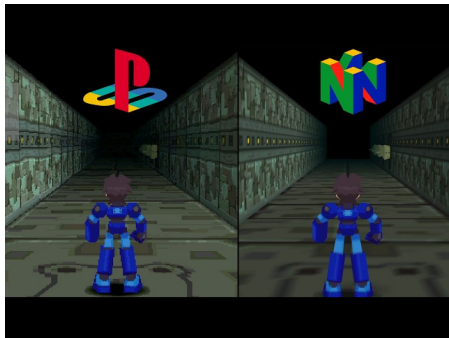
Since we only have data about the vertices of our 3D object, the natural (and really only) thing to do is to interpolate its coordinates.

For a pixel with barycentric coordinates $(\lambda_0, \lambda_1, \lambda_2)$ on screen, let's try:

$$u = \lambda_0 u_0 + \lambda_1 u_1 + \lambda_2 u_2.$$



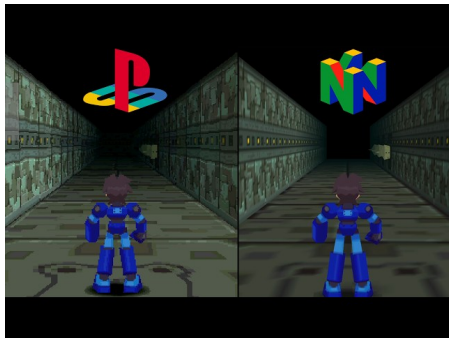
Issue with Linear Interpolation



The PlayStation 1 actually did exactly that (more specifically, they used an **affine** transformation) to map textures to the 3D shapes.



Issue with Linear Interpolation

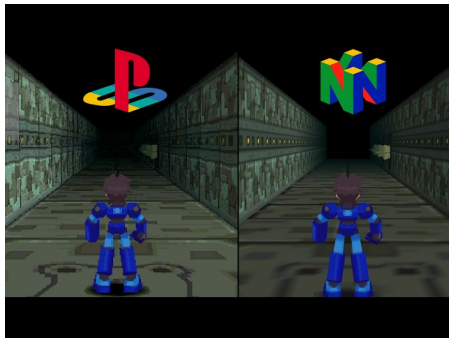


The PlayStation 1 actually did exactly that (more specifically, they used an **affine** transformation) to map textures to the 3D shapes.

But this assumes that something halfway across the screen is halfway across, say, the wall.



Issue with Linear Interpolation



The PlayStation 1 actually did exactly that (more specifically, they used an **affine** transformation) to map textures to the 3D shapes.

But this assumes that something halfway across the screen is halfway across, say, the wall.

It is not. Remember that projection does **not** preserve ratios along a line. And so as a surface tilted away from the camera, the texture did not adjust correctly for perspective, and floor and wall textures appeared to be **wobbly**.



Cross Ratios

Remember that perspective does not preserve ordinary ratios along a line, nor do they preserve parallel lines, angles, areas, etc... However, there is one useful quantity which *is* preserved that we can use to our advantage.



Cross Ratios

Remember that perspective does not preserve ordinary ratios along a line, nor do they preserve parallel lines, angles, areas, etc... However, there is one useful quantity which *is* preserved that we can use to our advantage.

Cross Ratio

Given four collinear points A, B, C, D , define

$$(A, B; C, D) = \frac{AC \cdot BD}{AD \cdot BC}.$$



Cross Ratios

Remember that perspective does not preserve ordinary ratios along a line, nor do they preserve parallel lines, angles, areas, etc... However, there is one useful quantity which *is* preserved that we can use to our advantage.

Cross Ratio

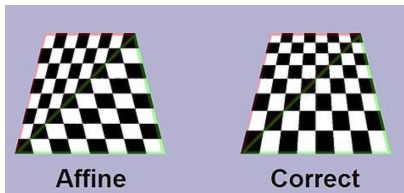
Given four collinear points A, B, C, D , define

$$(A, B; C, D) = \frac{AC \cdot BD}{AD \cdot BC}.$$

For completeness: the cross ratio is invariant under projective transformations. We will not prove why here; the useful part is that it lets us correct the wobbling we just saw.



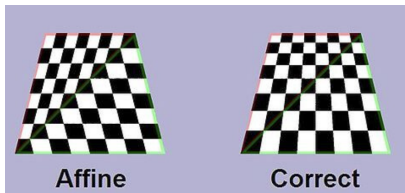
Perspective-Correct Interpolation



Linear interpolation on the **screen** is not linear interpolation on the **surface**.



Perspective-Correct Interpolation



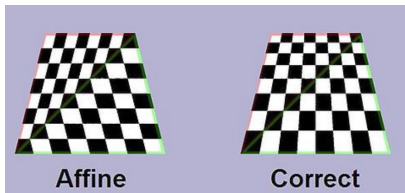
Linear interpolation on the **screen** is not linear interpolation on the **surface**.

Using the cross ratio, the full derivation reduces to a relationship of the form

$$t(s) = \frac{as + b}{cs + d}.$$



Perspective-Correct Interpolation



Linear interpolation on the **screen** is not linear interpolation on the **surface**.

Using the cross ratio, the full derivation reduces to a relationship of the form

$$t(s) = \frac{as + b}{cs + d}.$$

So the quantity we want is a **ratio of linear things**, not a linear thing itself.



Fragment Shader



Adding Detail

We now have the capability of building objects from triangle meshes and drawing them, however they lack any form of textures beyond flat colours.



Adding Detail

We now have the capability of building objects from triangle meshes and drawing them, however they lack any form of textures beyond flat colours.

To simply store colours at the vertices themselves would take a lot of overhead, and would require an immense amount of data to get any sense of realism (since the number of vertices in our model is now proportional to how many colours it needs).



Adding Detail

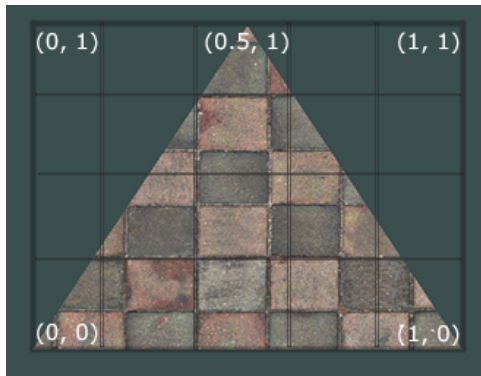
We now have the capability of building objects from triangle meshes and drawing them, however they lack any form of textures beyond flat colours.

To simply store colours at the vertices themselves would take a lot of overhead, and would require an immense amount of data to get any sense of realism (since the number of vertices in our model is now proportional to how many colours it needs).

So instead artists / programmers generally prefer to use a **texture**, a 2D image which acts as a sort of “wallpaper” that wraps around the 3D model to **simulate** detail without actually adding extra vertices.



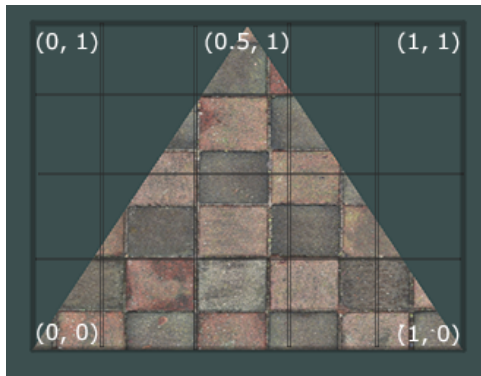
Texture Mapping



In order to map a texture to the triangle we need to tell each vertex of the triangle which part of the texture it corresponds to. Each vertex should thus have a texture coordinate associated with them that specifies what part of the texture image to sample from.



Texture Mapping



In order to map a texture to the triangle we need to tell each vertex of the triangle which part of the texture it corresponds to. Each vertex should thus have a texture coordinate associated with them that specifies what part of the texture image to sample from.

These coordinates are called **UV coordinates** (since x and y are already taken). And the process of assigning those coordinates is called **UV unwrapping**.



UV Unwrapping

Assigning a (u, v) to every vertex means laying the whole surface out flat on the image (think of peeling an orange and pressing the skin onto a table).



UV Unwrapping

Assigning a (u, v) to every vertex means laying the whole surface out flat on the image (think of peeling an orange and pressing the skin onto a table).

Can we see any issues with this?



UV Unwrapping

Assigning a (u, v) to every vertex means laying the whole surface out flat on the image (think of peeling an orange and pressing the skin onto a table).

Can we see any issues with this?

A curved surface like a sphere cannot be flattened onto a plane without introducing **some distortion**, hence every map of the Earth is inaccurate in some way.



UV Unwrapping

Assigning a (u, v) to every vertex means laying the whole surface out flat on the image (think of peeling an orange and pressing the skin onto a table).

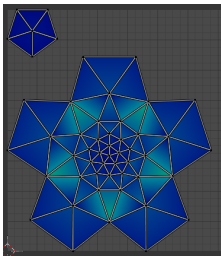
Can we see any issues with this?

A curved surface like a sphere cannot be flattened onto a plane without introducing **some distortion**, hence every map of the Earth is inaccurate in some way.

There is some lovely differential geometry explaining exactly why this must happen, (**Gauss's Theorema Egregium**), but for rendering the question comes down to **where we want to put the distortion**.

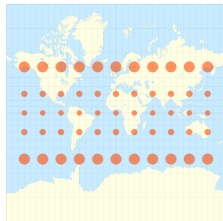


Pick Your Poison



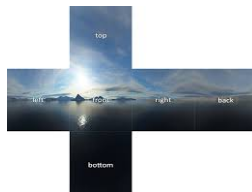
Cut it up

Introduce seams so each piece can lie flat. Flexible, but seams and stretching still exist.



Project it

Choose a cartographic-style projection. But one aspect is usually very distorted (e.g. area in the case of mercator).



Sidestep entirely

Sample textures using direction or several planar projections instead of one global unwrap.



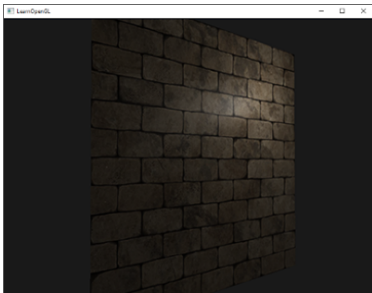
Doing more with maps

Unwrapping 3D objects to 2D maps lets us afford far more detail in our renders.
Consider this brick texture:



Doing more with maps

Unwrapping 3D objects to 2D maps lets us afford far more detail in our renders. Consider this brick texture:

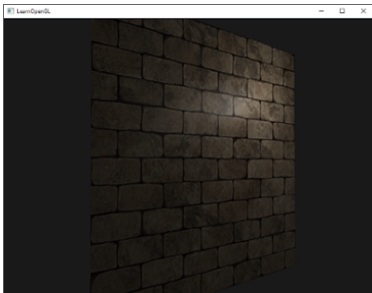


The lighting doesn't take into account the cracks, holes and deep grooves between the bricks, but modelling that geometry directly would be far too expensive.



Doing more with maps

Unwrapping 3D objects to 2D maps lets us afford far more detail in our renders. Consider this brick texture:



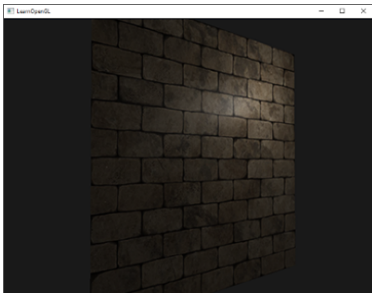
The lighting doesn't take into account the cracks, holes and deep grooves between the bricks, but modelling that geometry directly would be far too expensive.

Lighting depends almost entirely on the surface normal, but since surface has a *single* normal vector, it is lit uniformly.



Doing more with maps

Unwrapping 3D objects to 2D maps lets us afford far more detail in our renders. Consider this brick texture:



The lighting doesn't take into account the cracks, holes and deep grooves between the bricks, but modelling that geometry directly would be far too expensive.

Lighting depends almost entirely on the surface normal, but since surface has a *single* normal vector, it is lit uniformly.

What if we used a **per-fragment** normal instead of a per-surface one?



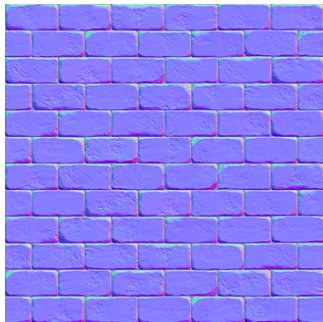
Normal Map

A texture stores three numbers per pixel (reserved for the r, g, b values). So to store vector information, we simply reinterpret r, g, b as the x, y, z of a normal, giving us a **normal map**.



Normal Map

A texture stores three numbers per pixel (reserved for the r, g, b values). So to store vector information, we simply reinterpret r, g, b as the x, y, z of a normal, giving us a **normal map**.



Almost every normal map is blue. Most normals point straight out of the surface, $\mathbf{n} \approx (0, 0, 1)$, which encodes to $(0.5, 0.5, 1)$.

Greener patches tilt towards $+y$, redder ones towards $+x$.



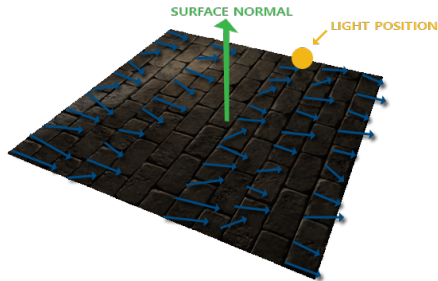
The Problem

Say we have a working normal map for a standing wall. Do its stored normals change when we lay the wall flat?



The Problem

Say we have a working normal map for a standing wall. Do its stored normals change when we lay the wall flat?

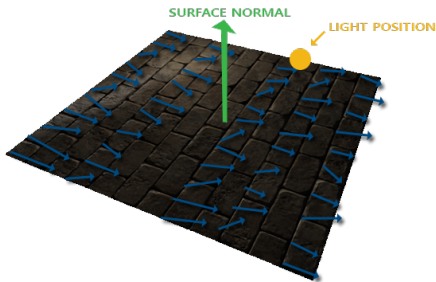


They do not. The normals still point along $+z$, so the wall is lit as though it were still standing up.



The Problem

Say we have a working normal map for a standing wall. Do its stored normals change when we lay the wall flat?



They do not. The normals still point along $+z$, so the wall is lit as though it were still standing up.

So instead of storing normals in world coordinates, we store them in a frame **attached to the surface**.



Tangent Space

At each point we build an orthonormal basis:

- **N**: the true surface normal;



Tangent Space

At each point we build an orthonormal basis:

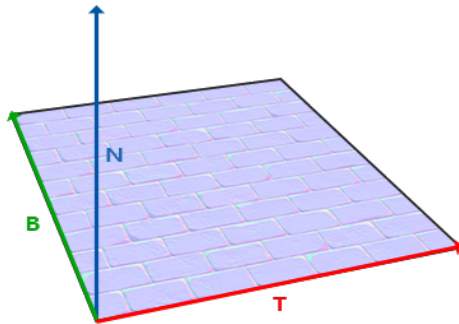
- **N**: the true surface normal;
- **T**: the **tangent**, along increasing u coordinates on the texture;



Tangent Space

At each point we build an orthonormal basis:

- **N**: the true surface normal;
- **T**: the **tangent**, along increasing u coordinates on the texture;
- **B**: the **bitangent**, along increasing v coordinates on the texture.

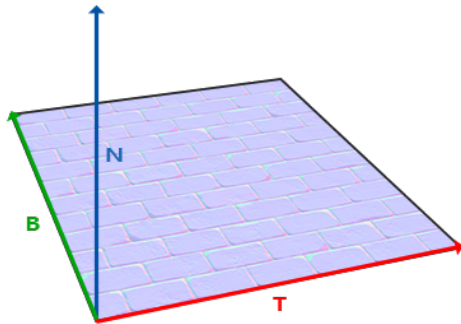




Tangent Space

At each point we build an orthonormal basis:

- **N**: the true surface normal;
- **T**: the **tangent**, along increasing u coordinates on the texture;
- **B**: the **bitangent**, along increasing v coordinates on the texture.



The normal map is expressed in *this* basis, and combining the three vectors then gives us a change of base matrix:

$$\text{TBN} = \begin{bmatrix} \mathbf{T} & \mathbf{B} & \mathbf{N} \end{bmatrix}, \quad \mathbf{n}_{\text{world}} = \text{TBN} \mathbf{n}_{\text{tangent}}.$$



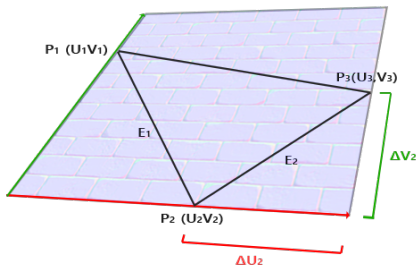
Calculating Tangent Space

We already know $\mathbf{N}$ as the surface normal. The tangent $\mathbf{T}$ and bitangent $\mathbf{B}$ come from how the **texture coordinates** runs along the triangle.



Calculating Tangent Space

We already know **N** as the surface normal. The tangent **T** and bitangent **B** come from how the **texture coordinates** runs along the triangle.



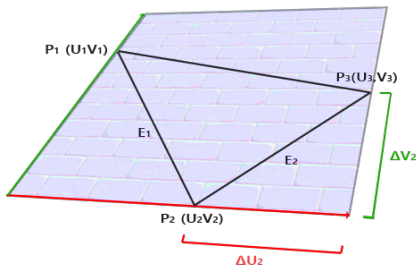
Since we are applying a texture on top of each triangle. Each triangle edge can be written as

$$\mathbf{E}_i = \Delta u_i \mathbf{T} + \Delta v_i \mathbf{B}.$$



Calculating Tangent Space

We already know **N** as the surface normal. The tangent **T** and bitangent **B** come from how the **texture coordinates** runs along the triangle.



Since we are applying a texture on top of each triangle. Each triangle edge can be written as

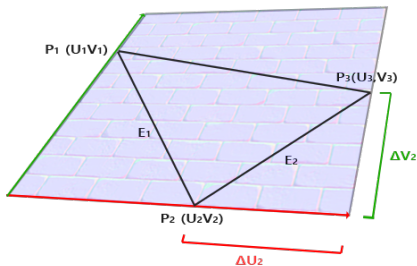
$$\mathbf{E}_i = \Delta u_i \mathbf{T} + \Delta v_i \mathbf{B}.$$

Write this for two edges of the triangles and we get **two linear equations** for the two unknown vectors **T** and **B**.



Calculating Tangent Space

We already know $\mathbf{N}$ as the surface normal. The tangent $\mathbf{T}$ and bitangent $\mathbf{B}$ come from how the **texture coordinates** runs along the triangle.



Since we are applying a texture on top of each triangle. Each triangle edge can be written as

$$\mathbf{E}_i = \Delta u_i \mathbf{T} + \Delta v_i \mathbf{B}.$$

Write this for two edges of the triangles and we get **two linear equations** for the two unknown vectors $\mathbf{T}$ and $\mathbf{B}$.

Finding $[\mathbf{T} \ \mathbf{B} \ \mathbf{N}]$

This is just a 2×2 linear system, which we can solve to build the change-of-basis matrix needed by the normal map. The explicit inverse is in the appendix.

That's All Folks

We've got an (acceptable) cube on screen. What do we do next?

Topic

Maths to explore

Lighting (Phong shading, PBR)

Dot products, reflections, the rendering equation

Face culling & depth buffers

Signed area / orientation again, interpolating $1/z$

Shadows

A second projection, from the light's point of view

Rotations & animation

Quaternions, splines and Bézier curves

Anti-aliasing & mipmaps

Sampling theory, Nyquist, Fourier transforms

Ray & path tracing

Ray-surface intersection, Monte Carlo integration

Post-processing (blur, bloom)

Convolution kernels

Good places to start: *learnopengl.com* and *pbr-book.org* (free).

Appendix: Optional Detail

Appendix / If We Have Time

These slides contain derivations and implementation details that are useful for completeness, but are not part of the default two-hour route through the talk.

Since the homogeneous coordinates $(x : y : z)$ are simply the ratios of these areas:

$$I = ([IBC] : [ICA] : [IAB]) = \left(\frac{1}{2}ar : \frac{1}{2}br : \frac{1}{2}cr \right)$$

Dividing out the common factor of $\frac{1}{2}r$ gives a very pretty result:

$$I = (a : b : c)$$

De Wobbling

Take a straight line on the wall, say the bottom edge of a floor tile running away from us.

De Wobbling

Take a straight line on the wall, say the bottom edge of a floor tile running away from us.

Let t measure its position along the **wall**, and let s measure position along its **image on screen**. Let's see if we can use the Cross Ratio to establish a relation between the two.

De Wobbling

Take a straight line on the wall, say the bottom edge of a floor tile running away from us.

Let t measure its position along the **wall**, and let s measure position along its **image on screen**. Let's see if we can use the Cross Ratio to establish a relation between the two.

Cross ratio needs four points, so we'll pick them as follows:

- A and B , the two endpoints of the line, these are triangle vertices, so we know their texture coordinates already;

De Wobbling

Take a straight line on the wall, say the bottom edge of a floor tile running away from us.

Let t measure its position along the **wall**, and let s measure position along its **image on screen**. Let's see if we can use the Cross Ratio to establish a relation between the two.

Cross ratio needs four points, so we'll pick them as follows:

- A and B , the two endpoints of the line, these are triangle vertices, so we know their texture coordinates already;
- R , any third fixed point between them, purely as a reference;

De Wobbling

Take a straight line on the wall, say the bottom edge of a floor tile running away from us.

Let t measure its position along the **wall**, and let s measure position along its **image on screen**. Let's see if we can use the Cross Ratio to establish a relation between the two.

Cross ratio needs four points, so we'll pick them as follows:

- A and B , the two endpoints of the line, these are triangle vertices, so we know their texture coordinates already;
- R , any third fixed point between them, purely as a reference;
- X , the point we are trying to locate, its position is t , which is unknown.

De Wobbling

Take a straight line on the wall, say the bottom edge of a floor tile running away from us.

Let t measure its position along the **wall**, and let s measure position along its **image on screen**. Let's see if we can use the Cross Ratio to establish a relation between the two.

Cross ratio needs four points, so we'll pick them as follows:

- A and B , the two endpoints of the line, these are triangle vertices, so we know their texture coordinates already;
- R , any third fixed point between them, purely as a reference;
- X , the point we are trying to locate, its position is t , which is unknown.

Each is drawn somewhere on screen; write A' , B' , R' , X' for the images, so X' sits at screen position s .

A Ratio of Linear Things

Projection preserves the cross ratio, so

$$(A, B; R, X) = (A', B'; R', X'),$$

A Ratio of Linear Things

Projection preserves the cross ratio, so

$$(A, B; R, X) = (A', B'; R', X'),$$

Subbing into the formula and simplifying leaves an equation in the form:

$$\alpha ts + \beta t + \gamma s + \delta = 0.$$

A Ratio of Linear Things

Projection preserves the cross ratio, so

$$(A, B; R, X) = (A', B'; R', X'),$$

Subbing into the formula and simplifying leaves an equation in the form:

$$\alpha ts + \beta t + \gamma s + \delta = 0.$$

Collecting the t terms and solving:

$$t = \frac{as + b}{cs + d}.$$

A Ratio of Linear Things

Projection preserves the cross ratio, so

$$(A, B; R, X) = (A', B'; R', X'),$$

Subbing into the formula and simplifying leaves an equation in the form:

$$\alpha ts + \beta t + \gamma s + \delta = 0.$$

Collecting the t terms and solving:

$$t = \frac{as + b}{cs + d}.$$

This means that t can never be a linear function of s , however we choose our endpoints.

A Ratio of Linear Things

Projection preserves the cross ratio, so

$$(A, B; R, X) = (A', B'; R', X'),$$

Subbing into the formula and simplifying leaves an equation in the form:

$$\alpha t s + \beta t + \gamma s + \delta = 0.$$

Collecting the t terms and solving:

$$t = \frac{as + b}{cs + d}.$$

This means that t can never be a linear function of s , however we choose our endpoints.

It is instead a ratio of two linear functions. So rather than interpolating t itself, we interpolate the numerator and denominator separately, and divide at the end.

Solving for the Frame

Writing these linear equations in matrix form gives us:

$$\begin{bmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{bmatrix} = \begin{bmatrix} \Delta u_1 & \Delta v_1 \\ \Delta u_2 & \Delta v_2 \end{bmatrix} \begin{bmatrix} \mathbf{T} \\ \mathbf{B} \end{bmatrix}$$

where each row is a 3-vector.

Solving for the Frame

Writing these linear equations in matrix form gives us:

$$\begin{bmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{bmatrix} = \begin{bmatrix} \Delta u_1 & \Delta v_1 \\ \Delta u_2 & \Delta v_2 \end{bmatrix} \begin{bmatrix} \mathbf{T} \\ \mathbf{B} \end{bmatrix}$$

where each row is a 3-vector.

Then we can simply find $\mathbf{T}$ and $\mathbf{B}$ by inverting the matrix:

$$\begin{bmatrix} \mathbf{T} \\ \mathbf{B} \end{bmatrix} = \frac{1}{\Delta u_1 \Delta v_2 - \Delta u_2 \Delta v_1} \begin{bmatrix} \Delta v_2 & -\Delta v_1 \\ -\Delta u_2 & \Delta u_1 \end{bmatrix} \begin{bmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{bmatrix}$$

We can now construct the *TBN* matrix, which can be used to convert our tangent space coordinates into world coordinates.

Solving for the Frame

Writing these linear equations in matrix form gives us:

$$\begin{bmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{bmatrix} = \begin{bmatrix} \Delta u_1 & \Delta v_1 \\ \Delta u_2 & \Delta v_2 \end{bmatrix} \begin{bmatrix} \mathbf{T} \\ \mathbf{B} \end{bmatrix}$$

where each row is a 3-vector.

Then we can simply find $\mathbf{T}$ and $\mathbf{B}$ by inverting the matrix:

$$\begin{bmatrix} \mathbf{T} \\ \mathbf{B} \end{bmatrix} = \frac{1}{\Delta u_1 \Delta v_2 - \Delta u_2 \Delta v_1} \begin{bmatrix} \Delta v_2 & -\Delta v_1 \\ -\Delta u_2 & \Delta u_1 \end{bmatrix} \begin{bmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{bmatrix}$$

We can now construct the TBN matrix, which can be used to convert our tangent space coordinates into world coordinates.

Note: Since T, B, N are orthonormal vectors, the TBN^{-1} can be easily calculated by taking its transpose, which is useful for lighting purposes (but we won't go into detail).